IRQ Priorities Overview

Overview & Implementation Steps

IRQ Priorities Overview

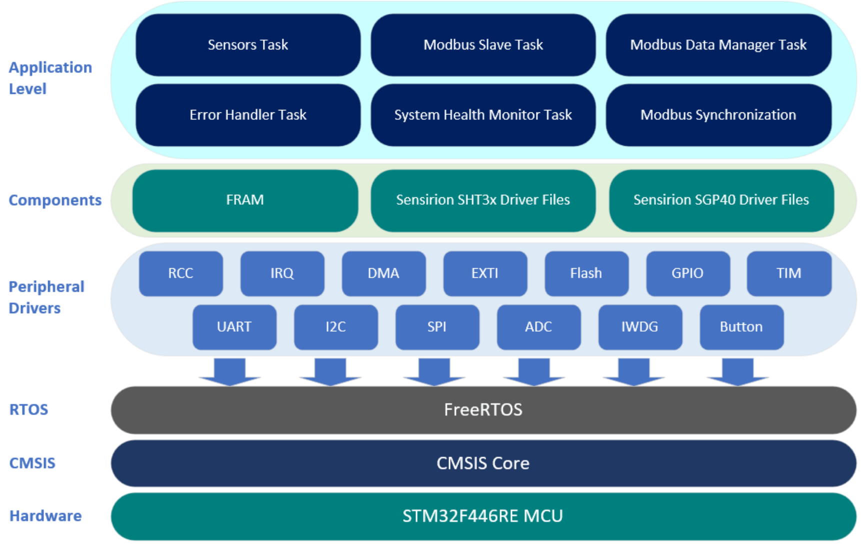
- IRQ Priorities Overview (Project-Specific)
- IRQ Importance in System Health Monitor (Reminder)
- Importance of IRQ Priorities
- IRQ Priorities in FreeRTOS
- FreeRTOS Configuration for IRQ Priorities
- Practical Implementation Steps

IRQ Priorities Overview (Project-Specific)

Overview

- Purpose & Functionality: **IRQ priorities ensure compatibility with FreeRTOS** by allowing safe use of FreeRTOS APIs within interrupts. **Critical for managing the timing and order of interrupt handling**, which impacts the reliability and performance of various system functions.

- Project Significance:
  - **ADC**: Internal temperature sensor.
  - **EXTI**: USER Button interrupts.
  - **I2C1 & TIM3**: Sensirion Sensor Drivers.
  - **USART2**: Modbus communication.
  - **SPI1 & DMA2/3**: FRAM operations.

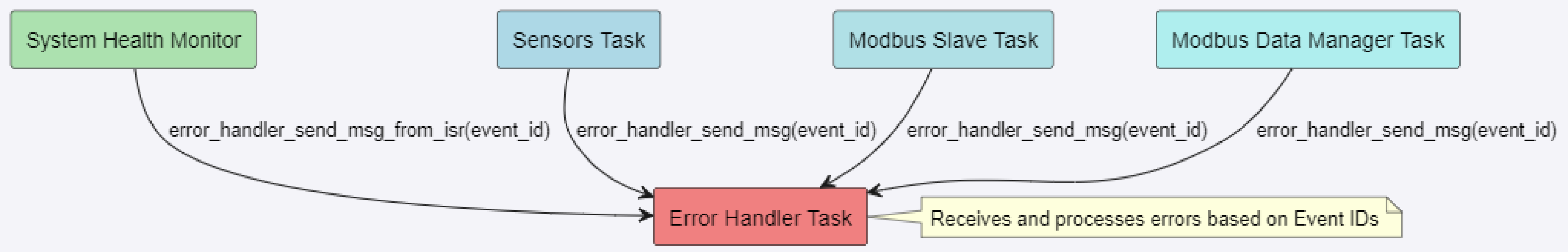


IRQ Importance in System Health Monitor (Reminder)

System Health Monitor Temperature Alarms: Communication facilitated through error_handler_send_msg_from_isr, via the ADC _IRQHandler Analog Watchdog Interrupts.

- **Analog Watchdog**: Monitors converted analog values against high and low thresholds. Triggers an interrupt if values fall outside these limits, detecting abnormal temperature conditions.
- **Communication Mechanism**: ADC_IRQHandler calls error_handler_send_msg_from_isr with EVT_SYS_HEALTH_AWDG_THRESHOLD_EXCEEDED event ID to notify the Error Handler Task.

- Project Significance: Ensures the System Health Monitor has immediate awareness of temperature threshold violations and communicates alarms to the Error Handler.



Importance of IRQ Priorities

Overview

- **Definition**: IRQ (Interrupt Request) priorities determine the order in which interrupts are processed.

- **Importance**: Properly setting IRQ priorities ensures that the most critical tasks are handled first, improving system reliability and performance.

Key Points

- **Nested Vectored Interrupt Controller (NVIC)**: Manages IRQ priorities in Cortex-M microcontrollers.

- **FreeRTOS and IRQs**: FreeRTOS requires careful management of IRQ priorities to ensure safe API calls within interrupt service routines (ISRs).

IRQ Priorities in FreeRTOS

Overview

- **Priority Levels**: Cortex-M microcontrollers typically support 4 to 16 priority levels, STM32F4XX uses 4 bits for Priority Levels.
- **Priority Bits**: Defined by **configPRIO_BITS** (typically 4 bits, giving 16 priority levels).

Key Points

- **configLIBRARY_LOWEST_INTERRUPT_PRIORITY**: The lowest priority level (e.g., 0xF).
- **configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY**: The highest priority level from which FreeRTOS API functions can be called (e.g., 5).

FreeRTOS Configuration for IRQ Priorities

FreeRTOSConfig.h Definitions

- **configPRIO_BITS**: Defines the number of priority bits implemented in the NVIC.
- **configLIBRARY_LOWEST_INTERRUPT_PRIORITY**: Set to the lowest priority (e.g., 0xF).
- **configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY**: Set to the highest priority from which FreeRTOS APIs can be called (e.g., 5).

Macro Calculations

- **configKERNEL_INTERRUPT_PRIORITY**: Set to the lowest priority.
- **configMAX_SYSCALL_INTERRUPT_PRIORITY**: Calculated to define the highest API-callable priority.

Practical Implementation Steps

Steps to Set IRQ Priorities

- **Define Priorities in the Header File**: #define IRQ_ADC_PRIORITY ((uint32_t) (configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY + 0))

- **Configure NVIC in Source File**:

```
void irq_set_priorities(void)
{
    NVIC_SetPriority(ADC_IRQn, IRQ_ADC_PRIORITY);
}
```

Importance

- Ensures Compatibility: Makes sure no higher-priority interrupt calls FreeRTOS APIs from an ISR.

Next: IRQ Header File